

Embeddings Practice Exercises Solutions

Name: Sorna parvathi. M

Register No: 99230041128

Department: CSE (Fourth Year)

Overview

This document contains the complete solutions and code implementations for the student practice exercises on generating and evaluating embeddings using Hugging Face models.

Setup & Initialization

Run these import statements and install the required dependencies before executing any of the exercises:

```
!pip install -q sentence-transformers scikit-learn matplotlib pandas
```

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np
import pandas as pd
```

```
model = SentenceTransformer("all-MiniLM-L6-v2")
print("Model loaded successfully!")
```

Exercise 1: Custom Semantic Search over 10 Sentences

For this exercise, we define a custom list of 10 sentences spanning different topics (technology, daily life, nature, and science) to demonstrate how semantic search retrieves relevant text based on meaning rather than keyword matching.

```
documents = [
    "Artificial intelligence is transforming modern software engineering.",
    "Fresh coffee brewing in the morning brings immense joy.",
    "Python remains a top language for data science and machine learning.",
    "Mount Everest is the highest mountain peak on Earth.",
```

```

"Regular physical exercise keeps the cardiovascular system healthy.",
"Deep neural networks process complex patterns efficiently.",
"The Pacific Ocean is the largest and deepest of Earth's oceanic divisions.",
"Balanced nutrition and adequate sleep are vital for wellness.",
"A gentle breeze rustled through the green pine trees.",
"Quantum computing promises revolutionary leaps in computational power."
]

# Encode the document database once
doc_embeddings = model.encode(documents, normalize_embeddings=True)

def semantic_search(query, top_k=3):
    query_embedding = model.encode([query], normalize_embeddings=True)
    scores = cosine_similarity(query_embedding, doc_embeddings)[0]
    ranked_idx = np.argsort(scores)[::-1][:top_k]

    print(f"Query: {query}")
    print("-" * 60)
    for rank, idx in enumerate(ranked_idx, start=1):
        print(f"{rank}. Score: {scores[idx]:.3f} | {documents[idx]}")
    print()

# Test queries
semantic_search("Tell me about machine learning and coding")
semantic_search("How do I maintain good health?")
semantic_search("Show me facts about mountains and oceans")

```

Exercise 2: Comparing MiniLM with MPNet

Here we compare the lightweight `all-MiniLM-L6-v2` model with the higher-accuracy `all-mpnet-base-v2` model. MPNet offers a larger embedding dimension (768 vs 384) and stronger semantic discrimination, though it requires more processing time.

```

model_mini = SentenceTransformer("all-MiniLM-L6-v2")
model_mpnet = SentenceTransformer("all-mpnet-base-v2")

emb_mini = model_mini.encode(documents, normalize_embeddings=True)
emb_mpnet = model_mpnet.encode(documents, normalize_embeddings=True)

def compare_models(query, top_k=2):
    print(f"QUERY: {query}")

```

```

for name, m, emb in [("MiniLM", model_mini, emb_mini), ("MPNet", model_mpnet,
emb_mpnet)]:
    q = m.encode([query], normalize_embeddings=True)
    scores = cosine_similarity(q, emb)[0]
    best_idx = scores.argmax()
    print(f" [{name}] Score: {scores[best_idx]:.3f} | {documents[best_idx]}")
    print("-" * 60)

compare_models("coding and programming languages")
compare_models("ocean depths and peaks")

```

Exercise 3: Building a Simple FAQ Bot with Confidence Thresholding

This exercise implements a functional FAQ assistant. It embeds customer questions rather than answers, computes similarity scores against user queries, and applies a confidence threshold to safely reject out-of-scope inquiries.

```

faq = [
    {"q": "What are your operating hours?", "a": "We are open Monday to Friday, 9:00 AM to 5:00 PM."},
    {"q": "How can I reset my account password?", "a": "Click 'Forgot Password' on the login screen to receive a reset link."},
    {"q": "What is the return policy?", "a": "Items can be returned within 30 days of purchase with a valid receipt."},
    {"q": "How do I track my shipment?", "a": "A tracking link is emailed to you once your order ships."},
    {"q": "How do I contact technical support?", "a": "You can reach support via email at help@example.com."}
]

```

```

faq_questions = [item["q"] for item in faq]
faq_embeddings = model.encode(faq_questions, normalize_embeddings=True)

```

THRESHOLD = 0.40

```

def ask_bot(user_question):
    q_emb = model.encode([user_question], normalize_embeddings=True)
    scores = cosine_similarity(q_emb, faq_embeddings)[0]
    best = int(np.argmax(scores))
    best_score = float(scores[best])

```

```
print(f"You asked: {user_question}")
if best_score < THRESHOLD:
    print("Bot: Sorry, I could not find a matching answer in our FAQ. Please contact support.")
else:
    print(f"Bot: {faq[best]['a']}")
    print(f"(Matched question: \"{faq[best]['q']}\" | Confidence: {best_score:.3f})")
print("-" * 60)
```

```
# Test cases including a refusal query
ask_bot("When are you guys open?")
ask_bot("I lost my password, what should I do?")
ask_bot("What is the weather forecast for tomorrow?")
```